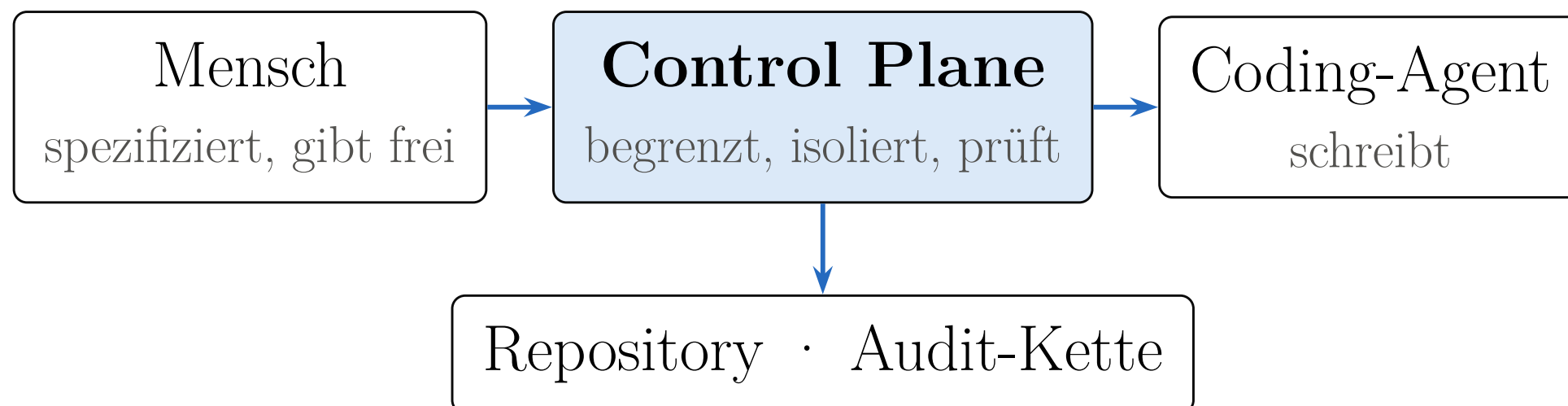


Der Engpass ist nicht das Modell. Es ist die fehlende Prozessschicht.

Coding-Agenten rufen Werkzeuge selbstständig auf. Das macht sie produktiv — und ihre Ausführung nichtdeterministisch und sicherheitsrelevant. Befunde der überarbeiteten Fassung, Stand August 2026.

Nicht möglichst viele Agenten. Eine Steuerschicht dazwischen.



Die Control Plane sitzt zwischen Mensch, Coding-Agent und Zielrepository. Sie begrenzt, isoliert, koordiniert, prüft und macht nachweisbar.

Ein Schreiber je Arbeitskopie. Mehrere unabhängige Prüfer.

Zwei Agenten, die gleichzeitig dieselbe Arbeitskopie verändern, erzeugen Schreibkonkurrenz, unklare Zurechnung und Zwischenstände, für die niemand mehr geradestehen kann.

Wer schreibt, gibt nicht frei.

Erzeugung und Freigabe sind technisch getrennt. Read-only-Reviewer liefern strukturierte Befunde — und ein Agent bewertet nie allein die eigene Arbeit.

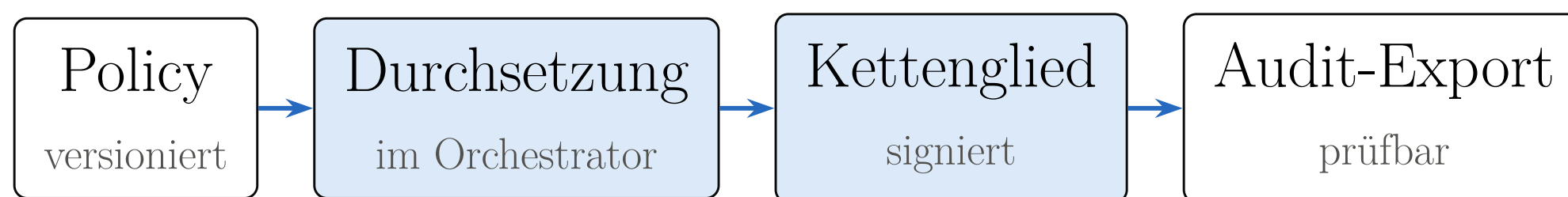
Ein Gate, dessen Ausfall wie Erfolg aussieht, ist schlimmer als kein Gate.

Ein ausgefallener Pflicht-Reviewer, eine nicht prüfbare Policy, eine fehlgeschlagene Attestierung: Nichts davon darf jemals als bestanden durchgehen.

Git trägt die Artefakte. Die Datenbank trägt den Prozesszustand.

Branches und Checkpoints gehören in Git. Workflow-, Policy-, Freigabe- und Audit-Zustand liegen autoritativ in der Datenbank — sonst ist der Prozess nicht rekonstruierbar.

**Für jeden Lauf rekonstruierbar:
welche Policy, welches Modell,
welche Freigabe.**



Jede Durchsetzung erzeugt ein signiertes Kettenglied in einer append-only Audit-Hashkette. Governance ist damit keine Ergänzung der Architektur, sondern Teil ihrer Struktur.

Vom Konzept zum System: 28 fachliche Slices, 27 Releases in vier Monaten.

Rund 33.200 Zeilen Produktivcode, Testabdeckung als buildbrechendes Gate (Line ab 85 %, Branch ab 81 %). Die Architektur des Whitepapers war die Grundlage — das Kapitel beschreibt, was daraus geworden ist. Stand: 6. August 2026, Release 0.20.0.

Parallelität entsteht aus Abhängigkeiten — nicht aus Agentenrollen.

Ein Feature wird in einen Task Graph zerlegt. Unabhängige Tasks sollen als isolierte Child Runs arbeiten — jeweils mit eigenem Branch oder Worktree und genau einem Schreiber. Erste Architekturvorbereitungen sind in Release 0.20.0 umgesetzt; produktiv nutzbare parallele Workflows, Merge Coordinator und Integration Gate sind geplant.

Agentic Software Development

Enterprise Architecture with AI Agents

Von kontrollierten Einzel-Runs zur parallelen Agentic Software Factory. Praxisbeispiele mit Java, Spring Boot und Domain-Driven Design

Vollständiges Whitepaper: Download-Link im Beitrag

janda.io · Überarbeitete und erweiterte Fassung 2.1, Stand: August 2026.